STA 5635 HW 5

Palmer Swanson

February 10, 2026

Problem 1

(1a) Loss function: $L(\boldsymbol{w}) = \frac{1}{N}\sum_{i=1}^{N} f(y_i \boldsymbol{x}_i^T \boldsymbol{w}) + s||\boldsymbol{w}||_2^2$, where $f(x) = \max(0, 1-x)$. I do not believe that the derivative of $f(x)$ exists, at least not cleanly. The portion of the loss consisting of the average of the function only contributes if the function doesn't return 0. In other words:

$$f(y_i \boldsymbol{x}_i^T \boldsymbol{w}) = \begin{cases} 0 & y_i \boldsymbol{x}_i^T \boldsymbol{w} \geq 1 \\ 1 - y_i \boldsymbol{x}_i^T \boldsymbol{w} & y_i \boldsymbol{x}_i^T \boldsymbol{w} < 1 \end{cases}$$

So for a point where $y_i \boldsymbol{x}_i^T \boldsymbol{w} < 1$, the gradient is $\frac{\partial}{\partial \boldsymbol{w}}(1 - y_i \boldsymbol{x}_i^T \boldsymbol{w}) = -y_i \boldsymbol{x}_i^T$. For a points where $y_i \boldsymbol{x}_i^T \boldsymbol{w} \geq 1$, the gradient is $\frac{\partial}{\partial \boldsymbol{w}}(0) = 0$. All together, the total gradient is:

$$\nabla_{\boldsymbol{w}} = \frac{-1}{N} \sum_{i:(y_i \boldsymbol{x}_i^T \boldsymbol{w}) < 1} (y_i \boldsymbol{x}_i^T) + 2s\boldsymbol{w}$$

We plot the training loss vs. iteration number is table 1.

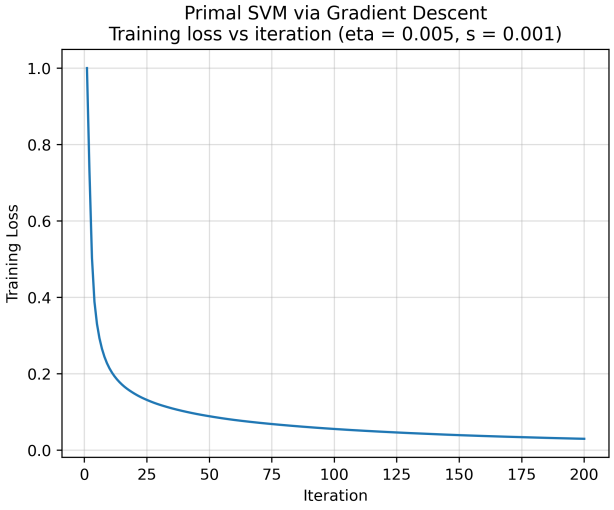

Figure 1: Primal SVM Training Loss vs. Iterations

The table outlining the training and testing misclassification errors is shown in figure 1:

Method	Train Misclass	Test Misclass
Primal SVM	0.006333	0.019000

Table 1: Train and Test Misclassification Errors, Primal SVM

Method	Train Misclass	Test Misclass	Num. Support Vectors
Linear SVM	0.000000	0.019000	1236

Table 2: Train, Test Misclassification Errors and Number of Support Vectors, Linear SVM, C=1

(1b) Use SVM package to train linear SVM, C=1. We can report the training errors and number of support vectors in table 2: Compared to part a, the test misclassification error is the same, but the train misclassification error is slightly lower.

(1c) Use SVM package to train a SVM with polynomial kernel of degree 2. We can report the training error and number of support vectors in table 3: Compared to part b, the

Method	Train Misclass	Test Misclass	Num. Support Vectors
Quadratic SVM	0.000167	0.025000	4744

Table 3: Train, Test Misclassification Errors and Number of Support Vectors, Quadratic SVM, C=1

number of support vectors is almost four times as great.

(1d) Train an SVM classifer with C=1 and RBF kernels $\gamma = 10^{-i}, i \in \{1, 2, \ldots, 6\}$. We plot the train and test misclassification errors vs γ in figure 2: We can also plot the number of support vectors vs γ, with a dashed line showing the number of support vectors in part b in figure 3:

(1e) To help with part d, we consider the following table 4:

Of all the models considered, it appears as though model a and model b both obtained the lowest test error of 0.019. Of models b through d, the model in part b (quadratic SVM) has the smallest number of support vectors (1236).

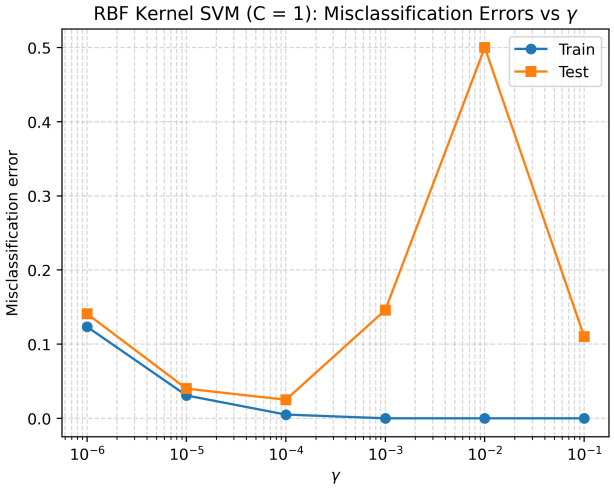

Figure 2: SVM Classifier, C=1, Train and Test Misclassification Errors vs γ.

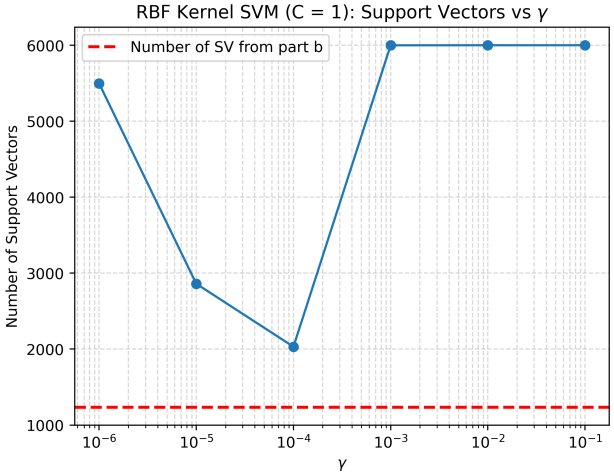

Figure 3: SVM Classifier, C=1, Number of Support Vectors vs γ. Dashed line indicates number of support vectors from part b (quadratic SVM).

γ	Train Error	Test Error	Support Vectors
0.100000	0.000000	0.110000	6000
0.010000	0.000000	0.500000	6000
0.001000	0.000000	0.146000	5999
0.000100	0.005000	0.025000	2031
0.000010	0.030833	0.040000	2860
0.000001	0.123500	0.141000	5497

Table 4: Train, Test Misclassification Errors and Number of Support Vectors, part d

Code

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
from sklearn import svm

#Q1 parameters.  Gradient Descent Iterations
iters = 200
s = 0.001
eta = 0.005

#get train and test
X_train = pd.read_csv("assignments/hw05/data/gisette_train.data",
    delim_whitespace=True, header=None)
X_valid = pd.read_csv("assignments/hw05/data/gisette_valid.data",
    delim_whitespace=True, header=None)

y_train = pd.read_csv("assignments/hw05/data/gisette_train.labels",
    header=None).values.ravel()
y_valid = pd.read_csv("assignments/hw05/data/gisette_valid.labels",
    header=None).values.ravel()

 #standardize
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_valid)

#add column of 1s to augment w
X_train_tilde = np.column_stack([np.ones(len(X_train_scaled)), X_train_scaled])
X_valid_tilde = np.column_stack([np.ones(len(X_test_scaled)), X_test_scaled])


#part a
#get N and p
N, p = X_train_tilde.shape

#initialize to all 0
w = np.zeros(p)

training_loss = []

for i in range(iters):
    if i % 50 == 0:
        print(i)
    #don't need transpose?
```

```
    #y_i*x_i*w
    pred = y_train*(X_train_tilde @ w)
    #the f(x) portion
    fun = np.maximum(0.0, 1.0 - pred)

    #loss function is mean of fun with the penalty term
    #dot product of w.  Don't penalize intercept
    loss = np.mean(fun) + s*(np.dot(w[1:], w[1:]))

    training_loss.append(loss)

    #gradient
    #select points in form of Boolean vector.  T for being less than 1 and
    contributing, F for being 1 or greater
    #and not penalizing the intercept term (w0)
    contrib_points = pred < 1.0
    gradient = (-1.0/N)*(X_train_tilde[contrib_points].T @
    y_train[contrib_points])
    gradient[1:] = gradient[1:] + (2.0*s*w[1:])

    #update w
    w = w - (eta*gradient)

#make predictions.  Need a sign function
yhat_train = np.sign(X_train_tilde @ w)
yhat_test = np.sign(X_valid_tilde @ w)

train_misclass = np.mean(yhat_train != y_train)
test_misclass  = np.mean(yhat_test != y_valid)

#plot
plt.figure()
plt.plot(np.arange(1, iters + 1), training_loss)
plt.xlabel("Iteration")
plt.ylabel("Training␣Loss")
plt.title(
    f"Primal␣SVM␣via␣Gradient␣Descent\n"
    f"Training␣loss␣vs␣iteration␣(eta␣=␣{eta},␣s␣=␣{s})"
)
plt.grid(True, alpha=0.4)
plt.savefig("assignments/hw05/figures/primal_SVM_training_loss.png", dpi=400,
    bbox_inches="tight")
plt.close()
#plt.show()

#report the errors
```

```
#need to save this as a table.  Brackets around the elements works for some
    reason
#https://medium.com/practical-pandas/pandas-dictionary-to-dataframe-5-ways-to-convert-dictiona
error_df = pd.DataFrame({
    "Method": ["Primal␣SVM"],
    "Train␣Misclass": [train_misclass],
    "Test␣Misclass": [test_misclass]
})
print(error_df)

#need to send this table to LATEX
LATEX_table = error_df.to_latex(
    index=False,
    caption=None,
    label=None,
    column_format="c" * error_df.shape[1],
    escape=False
)

#save latex table
with open("assignments/hw05/output/train_test_errors.tex", "w") as f:
    f.write(LATEX_table)

#part b
#SVM
#https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html#sklearn.svm.Line
#doesn't report number of support vectors
#https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html#sklearn.svm.SVC
#does
svm_fit = svm.SVC(
    C=1.0,
    kernel="linear"
)

svm_fit.fit(X_train_scaled, y_train)

#make predictions
yhat_train = svm_fit.predict(X_train_scaled)
yhat_test  = svm_fit.predict(X_test_scaled)

#get errors and number of support vectors
train_err = np.mean(yhat_train != y_train)
test_err  = np.mean(yhat_test != y_valid)
num_sv = svm_fit.n_support_.sum()

#save the number of support vectors for later
saved_for_later = num_sv
```

```
131
132 error_df = pd.DataFrame({
133     "Method": ["Linear SVM"],
134     "Train Misclass": [train_err],
135     "Test Misclass": [test_err],
136     "Num. Support Vectors": [num_sv]
137 })
138 print(error_df)
139
140 #need to send this table to LATEX
141 LATEX_table = error_df.to_latex(
142     index=False,
143     caption=None,
144     label=None,
145     column_format="c" * error_df.shape[1],
146     escape=False
147 )
148
149 #save latex table
150 with open("assignments/hw05/output/train_test_errors_linear_svm.tex", "w") as f:
151     f.write(LATEX_table)
152
153
154 #part c, using polynomial kernel, degree 2
155 svm_poly = svm.SVC(
156     kernel="poly",
157     degree=2,
158     C=1.0
159 )
160
161 svm_poly.fit(X_train_scaled, y_train)
162
163 #make predictions
164 yhat_train = svm_poly.predict(X_train_scaled)
165 yhat_test  = svm_poly.predict(X_test_scaled)
166
167 #get error and number of support vectors
168 train_err = np.mean(yhat_train != y_train)
169 test_err  = np.mean(yhat_test != y_valid)
170 num_sv = svm_poly.n_support_.sum()
171
172 error_df = pd.DataFrame({
173     "Method": ["Quadratic SVM"],
174     "Train Misclass": [train_err],
175     "Test Misclass": [test_err],
176     "Num. Support Vectors": [num_sv]
177 })
```

```
178 print(error_df)
179
180 #need to send this table to LATEX
181 LATEX_table = error_df.to_latex(
182     index=False,
183     caption=None,
184     label=None,
185     column_format="c" * error_df.shape[1],
186     escape=False
187 )
188
189 #save latex table
190 with open("assignments/hw05/output/train_test_errors_polynomial_svm.tex", "w")
        as f:
191     f.write(LATEX_table)
192
193
194 #part d, RBF kernel estimators
195 train_errs = []
196 test_errs = []
197 num_sup_vector = []
198
199 #fill gammas with loop
200 gammas = []
201 for i in range(1,7):
202     temp = 10.0**(-i)
203     gammas.append(temp)
204
205 #looping over the gamma values
206 for gamma in gammas:
207     print(gamma)
208     svm_brf = svm.SVC(
209         kernel="rbf",
210         C=1.0,
211         gamma=gamma
212         )
213
214     svm_brf.fit(X_train_scaled, y_train)
215
216     # predictions
217     yhat_train = svm_brf.predict(X_train_scaled)
218     yhat_test  = svm_brf.predict(X_test_scaled)
219
220     # misclassification errors
221     train_err = np.mean(yhat_train != y_train)
222     test_err  = np.mean(yhat_test != y_valid)
223     num_sv = svm_brf.n_support_.sum()
```

```
224
225     #append for plotting and table
226     train_errs.append(train_err)
227     test_errs.append(test_err)
228     num_sup_vector.append(num_sv)
229
230 #saving these results as a df, with \gamma for ease with LATEX
231 partd_results = pd.DataFrame({
232     r"$\gamma$": gammas,
233     "Train␣Error": train_errs,
234     "Test␣Error": test_errs,
235     "Support␣Vectors": num_sup_vector
236 })
237
238 #need to send this table to LATEX
239 LATEX_table = partd_results.to_latex(
240     index=False,
241     caption=None,
242     label=None,
243     column_format="c" * partd_results.shape[1],
244     escape=False
245 )
246
247 #save latex table
248 with open("assignments/hw05/output/train_test_errors_num_sv_svm_partd.tex", "w")
        as f:
249     f.write(LATEX_table)
250
251 #plot these gamma values
252 plt.figure()
253 plt.semilogx(gammas, train_errs, marker="o", label="Train")
254 plt.semilogx(gammas, test_errs, marker="s", label="Test")
255
256 #can use r to get LATEX symbols
257 plt.xlabel(r"$\gamma$")
258 plt.ylabel("Misclassification␣error")
259 plt.title("RBF␣Kernel␣SVM␣(C␣=␣1):␣Misclassification␣Errors␣vs␣$\\gamma$")
260 plt.legend()
261 plt.grid(True, which="both", linestyle="--", alpha=0.5)
262 plt.savefig("assignments/hw05/figures/misclass_errors_v_gamma.png", dpi=400,
        bbox_inches="tight")
263 plt.close()
264 #plt.show()
265
266
267
268 #plot the number of support vectors vs gamma with horizontal line
```

```
plt.figure()
plt.semilogx(gammas, num_sup_vector, marker="o")
plt.axhline(
    y=saved_for_later,
    linestyle="--",
    linewidth=2,
    color="red",
    label=f"Number␣of␣SV␣from␣part␣b"
)
#can use r to get LATEX symbols
plt.xlabel(r"$\gamma$")
plt.ylabel("Number␣of␣Support␣Vectors")
plt.title("RBF␣Kernel␣SVM␣(C␣=␣1):␣Support␣Vectors␣vs␣$\\gamma$")
plt.legend()
plt.grid(True, which="both", linestyle="--", alpha=0.5)
plt.savefig("assignments/hw05/figures/num_support_vectors_v_gamma.png", dpi=400,
    bbox_inches="tight")
plt.close()
#plt.show()
```